Challenge 2 - Blockchain Security

Operation: Break the Robot Economy
Cybersecurity Red Team vs. Blue Team Challenge

1. Challenge Objective

Your team will act as both hackers (Red Team) and defenders (Blue Team). You must understand a blockchain-based payment system for delivery robots, find the flaws in its "smart contract," safely prove the vulnerabilities, fix the code, and verify that your fixes actually work.

2. The Scenario

A robotics company operates autonomous delivery robots around a university campus. Students and staff pay for deliveries using blockchain-based digital tokens.
A **Smart Contract** (the automated code that runs on the blockchain) manages user balances, processes payments to robots, registers deliveries, and handles refunds. Because this contract handles real money, a single bug could allow attackers to steal funds, freeze the system, or get free deliveries. The original developers focused on making the features work, but completely ignored security.

3. System Architecture

```
User App → Blockchain API → Smart Contract → Local Blockchain → Payment Confirmed → Robot Dispatched
```

4. Your Missions

- **Red-Team Mission (Week 1):** Find ways to break the contract. Can you pay with negative money? Can you steal admin controls? Can you request infinite refunds? Can you pause the system without permission?
- **Blue-Team Mission (Week 2):** Fix the contract. Implement strict access controls, validate all inputs, log all events transparently, and ensure the contract can't be tricked into giving out free money.

5. Two-Week Work Plan

Week	Focus	Main Activities	What You Must Produce
Week 1	**Analyze & Attack**	Understand the architecture; read the flawed contract code; find logic and security bugs; perform controlled "hacks" to prove the bugs exist.	Architecture map, Vulnerability Table, Screenshots/Logs of your successful hacks.
Week 2	**Defend & Verify**	Rewrite the contract to fix the bugs; add security features; run your Week 1 hacks again to prove they now fail; write the final report.	Corrected Code, Before/After comparison, Final Technical Report.

6. The Core Task: Debugging the Smart Contract

Below is the Python simulation of the Smart Contract. It is intentionally full of logic errors and severe security vulnerabilities.

`robot_economy_contract.py`

```
class RobotEconomyContract:
    def __init__(self, admin_address):
        self.admin = admin_address
        self.balances = {
            "alice": 1000,
            "bob": 500,
            "robot_01": 0,
            "robot_02": 0,
            "admin": 10000
        }
        self.deliveries = []
        self.is_paused = False
        self.events = []  # The devs created this but forgot to use it!

    # --- PAYMENT FUNCTION ---
    def pay_robot(self, caller, robot, amount):
        # TODO: Devs forgot to check if the contract is paused!
        # TODO: Devs forgot to check if amount is positive!
        # TODO: Devs forgot to check if caller has enough balance!
        # TODO: Devs forgot to check if the robot actually exists!

        # VULNERABILITY: Updating state before validating everything
        self.balances[caller] = self.balances[caller] - amount
        self.balances[robot] = self.balances[robot] + amount

        return {"status": "success", "message": f"{amount} tokens sent to
{robot}"}

    # --- REFUND FUNCTION ---
    def request_refund(self, caller, delivery_id, amount):
        # TODO: Devs forgot to verify if the delivery actually failed!
        # TODO: Devs forgot to check if the refund amount matches the
original payment!

        # VULNERABILITY: Free money exploit
        self.balances[caller] = self.balances[caller] + amount
        return {"status": "success", "message": "Refund processed"}

    # --- ADMIN FUNCTION ---
    def pause_contract(self, caller):
        # TODO: Devs forgot to check if the caller is the admin!
        self.is_paused = True
        return "Contract paused by emergency admin"

    # --- ADMIN FUNCTION ---
    def withdraw_emergency_funds(self, caller, amount):
```

```
        # TODO: Devs forgot to check if the caller is the admin!
        self.balances[caller] = self.balances[caller] + amount
        self.balances["admin"] = self.balances["admin"] - amount
        return "Emergency funds withdrawn"
```

7. Team Task Instructions

Part 1: Identify Logic & Robustness Errors Analyze the code for bugs that will cause the system to behave incorrectly.

1. What happens in `pay_robot` if a user sends a **negative** amount (e.g., `-500`)? What happens to their balance? What happens to the robot's balance?
2. What happens if a user tries to pay `10,000` tokens when they only have `1,000`? Does the system stop them?
3. What happens if a user tries to send money to a robot that doesn't exist (e.g., `"robot_99"`)?

Part 2: Identify Security Vulnerabilities (Smart Contract Flaws) Assume you are an attacker interacting with this contract.

1. **Access Control:** Look at `pause_contract` and `withdraw_emergency_funds`. Can any user call these functions? What is the impact?
2. **Infinite Refunds / Business Logic:** Look at `request_refund`. Can a user call this function multiple times for the same delivery? Can they ask for a refund of `999,999` tokens?
3. **Missing Event Logging:** Why is the `self.events` list empty? In blockchain, why is it critical to log every state change (payments, refunds)?
4. **Checks-Effects-Interactions:** In `pay_robot`, the balances are updated *before* any validation checks are made. Why is this a dangerous pattern in smart contracts?

Part 3: Deliverables In your final report, you must include:

1. **Vulnerability Table:** A list of all logic and security flaws found.
2. **Red-Team Evidence:** Proof of your exploits (e.g., "I called `request_refund` 5 times and turned my 500 balance into 5000").
3. **Corrected Code:** The fully rewritten, secure version of `robot_economy_contract.py`.
4. **Security Justification:** Explain why you added specific checks (e.g., "Added `require(caller == self.admin)` to prevent unauthorized pauses").

Lab Rule: All attacks must remain inside the provided Python simulations. Teams must not scan real networks, test real accounts, collect real passwords, or connect the programs to real smart devices, robots, UAVs, or external systems.